

Lab 09 - Expected verification evidence

Run from the lab directory, after the failure injection in section 6 and before the cleanup in section 8, with the mfs.io/maintenance taint still applied:

```
bash verification/checks.sh
```

Expected transcript (single-node kind cluster)

```
Lab 09 verification - namespace kcna-lab09

== Namespace and placement policy
[PASS] namespace kcna-lab09 is Active (= Active)
[PASS] placement policy read: 6 workload rows

== Cluster shape
[PASS] cluster has 1 node(s); first node is kind-control-plane

== Node labels required by the policy (apply_label=yes)
[PASS] node label mfs.io/zone (needed by gate-scanner) (= sg-harbourfront)
[PASS] node label mfs.io/disk (needed by manifest-store) (= ssd)

== Node taint applied for the maintenance-window exercise
[PASS] taint on kind-control-plane (= mfs.io/maintenance=window:NoSchedule)

== Pod placement outcomes match the policy
[PASS] gate-scanner (nodeSelector) phase (= Running)
[PASS] manifest-store (requiredNodeAffinity) phase (= Running)
[PASS] eta-predictor (preferredNodeAffinity) phase (= Running)
[PASS] draft-optimiser (requiredNodeAffinity) phase (= Pending)
[PASS] yard-report (none) phase (= Pending)
[PASS] crane-controller (toleration) phase (= Running)

== Scheduled Pods really landed on the node
[PASS] gate-scanner scheduled onto kind-control-plane (= kind-control-plane)
[PASS] manifest-store scheduled onto kind-control-plane (= kind-control-plane)
[PASS] eta-predictor scheduled onto kind-control-plane (= kind-control-plane)
[PASS] crane-controller scheduled onto kind-control-plane (= kind-control-plane)

== Pending Pods have no nodeName and a FailedScheduling event
[PASS] draft-optimiser has no assigned node (= )
[PASS] draft-optimiser PodScheduled reason (= Unschedulable)
[PASS] draft-optimiser has a FailedScheduling event: 0/1 nodes are available: 1 node(s)
didn't match Pod's node affinity/selector. preemption: 0/1 nodes are available: 1 Preemption
is not helpful for scheduling.
[PASS] yard-report has no assigned node (= )
[PASS] yard-report PodScheduled reason (= Unschedulable)
[PASS] yard-report has a FailedScheduling event: 0/1 nodes are available: 1 node(s) had
untolerated taint {mfs.io/maintenance: window}. preemption: 0/1 nodes are available: 1
Preemption is not helpful for scheduling.

== Mechanism is declared where the policy says it should be
[PASS] gate-scanner nodeSelector mfs.io/zone (= sg-harbourfront)
[PASS] manifest-store required affinity operators (= In DoesNotExist)
[PASS] eta-predictor preference weights (= 80 20)
[PASS] crane-controller tolerates NoSchedule (= NoSchedule)

-----
Lab 09: 26 passed, 0 failed
ALL CHECKS PASSED
```

```
echo $?
```

```
0
```

Supporting evidence the learner should be able to produce

1. The node was untainted to begin with - checked, not assumed

```
kubectl describe node kind-control-plane | grep -A1 '^Taints:'
```

Before Step 6:

```
Taints:          <none>
Unschedulable:   false
```

After Step 6:

```
Taints:          mfs.io/maintenance=window:NoSchedule
Unschedulable:   false
```

2. The unsatisfiable required affinity - the label exists nowhere

```
kubectl get nodes -l mfs.io/hardware=gpu-a100
```

```
No resources found
```

```
kubectl -n kcna-lab09 get pod draft-optimiser \
-o jsonpath='{.status.conditions[?(@.type=="PodScheduled")].message}'
```

```
0/1 nodes are available: 1 node(s) didn't match Pod's node affinity/selector.
preemption: 0/1 nodes are available: 1 Preemption is not helpful for scheduling.
```

3. The intolerated taint - a different predicate, same terminal state

```
kubectl -n kcna-lab09 describe pod yard-report | sed -n '/^Events:/, $p'
```

```
Events:
  Type            Reason              Age   From                  Message
  ----            -
  Warning         FailedScheduling    12s   default-scheduler    0/1 nodes are available: 1 node(s) had
  intolerated taint {mfs.io/maintenance: window}.
  preemption: 0/1 nodes are available: 1
  Preemption is not helpful for scheduling.
```

4. Preferred affinity did not block placement

eta-predictor asks for mfs.io/zone=sg-tuas at weight 80. That zone does not exist here, yet:

```
kubectl -n kcna-lab09 get pod eta-predictor -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	
NOMINATED NODE	READINESS	GATES					
eta-predictor	1/1	Running	0	7m	10.244.0.14	kind-control-plane	<none>
<none>							

5. Toleration vs no toleration, on the same tainted node

```
kubectl -n kcna-lab09 get pods -l 'placement in (no-toleration,toleration)' \
-o custom-columns=POD:.metadata.name,STATUS:.status.phase,NODE:.spec.nodeName,TOLERATES:.spec.tolerations[0].key
```

POD	STATUS	NODE	TOLERATES
crane-controller	Running	kind-control-plane	mfs.io/maintenance
yard-report	Pending	<none>	<none>

6. The node was returned to its original state after cleanup

```
kubectl get node kind-control-plane -o jsonpath='{.spec.taints}'; echo
kubectl get node kind-control-plane --show-labels | grep -c 'mfs.io' || echo "no mfs.io labels remain"
```

```
taints=
no mfs.io labels remain
```

Notes on variance

* Node name and count. The script reads the node name from the API and compares desired behaviour against it, so it is correct on any cluster. On a multi-node cluster you would label only one node and the other nodes' FailedScheduling tallies would read 0/N.

* Architecture. kubernetes.io/arch is arm64 on Apple Silicon and amd64 on x86 hosts. Nothing in this lab depends on it.

* Event expiry. Kubernetes garbage-collects events after roughly one hour. If a FailedScheduling check fails but PodScheduled=Unschedulable still passes, delete and re-apply the Pod to regenerate the event. The .status.conditions message never expires and carries the same text.

* Pod IPs (10.244.0.x) are assigned by the CNI and will differ.

* If you leave mfs.io/hardware=gpu-a100 on the node from Fix Option A, draft-optimiser will be Running and the policy check for that row will fail with expected 'Pending', got 'Running'. Remove the label and re-create the Pod.